

Data Quality and Validation in ETL Assignment

Question 1: Define Data Quality in the context of ETL pipelines. Why is it more than just data cleaning?

Answer:

Data Quality refers to the degree to which data is fit for its intended use across the entire ETL (Extract, Transform, Load) process ensuring data is accurate, consistent, complete, and reliable when it reaches the destination system.

Core Dimensions of Data Quality

Dimension Meaning

Accuracy: Data correctly represents real-world values

Completeness: No missing or null critical fields

Consistency: Same data looks the same across systems

Timeliness: Data is available when needed

Validity: Data conforms to defined formats/rules

Uniqueness: No duplicate records

Why It's More Than Just Data Cleaning

Data cleaning is reactive: it fixes dirty data after it's found.

Data Quality is (proactive and end-to-end), covering:

Extraction: Detecting corrupt/incomplete data at the source

Transformation: Applying business rules, validation checks, and standardization

Loading: Ensuring referential integrity and schema compliance at the target

Monitoring: Continuous profiling, alerting, and auditing post-load

Governance: Ownership, lineage tracking, and compliance (e.g., GDPR)

In short: Data cleaning removes bad data. Data Quality prevents bad data from ever impacting the pipeline and ensures trust in data at every stage.

Question 2: Explain why poor data quality leads to misleading dashboards and incorrect decisions.

Answer:

Data quality and decision-making are directly linked. When the data entering an ETL pipeline is inaccurate, incomplete, or inconsistent, the dashboards built on top of that data become unreliable, and any decisions made from those dashboards carry serious risk.

The core issue is that dashboards do not question the data they display. They simply aggregate and visualize whatever has been loaded into the system. So if the underlying data contains duplicates, missing values, or incorrect entries, the dashboard will present those errors as facts, often without any visible warning.

For example, if duplicate sales records are not removed during transformation, the total revenue figure on a dashboard will appear inflated. A manager looking at that figure has no reason to doubt it and may approve new hiring or expansion based on revenue that does not actually exist. Similarly, if customer records have missing demographic fields, a marketing dashboard may show skewed segmentation, leading to campaigns targeted at the wrong audience.

Outdated or stale data is another major issue. If the ETL pipeline fails to refresh data on time, the dashboard reflects an old state of the business. Decisions made on such dashboards, especially in fast-moving domains like inventory or finance, can be completely misaligned with the current reality.

The most dangerous aspect of poor data quality is not that it produces wrong numbers, but that it produces confidently wrong numbers. Unlike a system crash or an obvious error, a dashboard with bad data looks perfectly normal. Decision-makers trust it, present it in meetings, and act on it, all without knowing the foundation is flawed.

In conclusion, poor data quality does not just corrupt numbers. It corrupts the entire decision-making process by creating a false sense of accuracy, leading organizations toward strategies built on faulty evidence.

Question 3: What is duplicate data? Explain three causes in ETL pipelines.

Answer:

Duplicate data refers to the presence of two or more records in a dataset that represent the same real-world entity or event. These records may be completely identical or may differ slightly in formatting while still referring to the same person, transaction, or object. In ETL pipelines, duplicate data is one of the most common and damaging data quality issues because it silently inflates counts, totals, and aggregates without triggering any obvious error.

The first cause is multiple source system extractions. In many organizations, data is pulled from more than one source system simultaneously, such as a CRM, an ERP, and a third-party API. If the same customer or transaction exists in more than one of these systems and no deduplication logic is applied during extraction, the same record gets loaded into the destination multiple times. This is especially common in organizations that have gone

through mergers or system migrations where overlapping data exists across old and new platforms.

The second cause is pipeline re-runs without proper checkpointing. ETL pipelines sometimes fail midway due to network issues, system timeouts, or resource errors. When the pipeline is restarted without a mechanism to track which records were already successfully loaded, it processes and loads the same data again from the beginning. Without an idempotency check or a watermark-based incremental load strategy, every re-run adds duplicate rows to the destination table.

The third cause is lack of deduplication logic during transformation. Many ETL pipelines focus heavily on format conversion and business rule application but skip the step of checking for existing records before inserting new ones. If the transformation layer does not include distinct checks, primary key validation, or merge logic such as upsert operations, identical or near-identical records pass through unchallenged and accumulate in the target database over time.

In summary, duplicate data in ETL pipelines is rarely accidental in isolation. It results from gaps in pipeline design, specifically in how data is extracted from multiple sources, how failures are handled during re-runs, and how the transformation stage manages record uniqueness before loading.

Question 4: Differentiate between exact, partial, and fuzzy duplicates.

Answer:

In the context of data quality and ETL pipelines, not all duplicate records look the same. Duplicates are broadly classified into three types based on how closely the repeated records resemble each other. Understanding these distinctions is important because each type requires a different detection and resolution strategy.

Exact duplicates are records where every single field is identical across two or more rows. Nothing differs, not the values, not the formatting, not the spacing. These are the easiest type of duplicate to detect because a simple comparison of all column values will immediately flag them. They typically occur when the same data file is loaded twice or when a pipeline re-runs without checkpointing. A straightforward distinct query or primary key constraint is usually sufficient to identify and remove them.

Partial duplicates are records that share the same core identity but differ in one or more fields. For example, two rows may represent the same customer but one has a phone number filled in while the other does not, or one has an updated address while the other still carries the old one. These are more difficult to handle than exact duplicates because a simple comparison will not flag them as duplicates. They require business logic to determine which record is more complete or more recent, and then a merge or upsert operation to consolidate them into a single authoritative record.

Fuzzy duplicates are the most complex type. These are records that represent the same real-world entity but contain variations due to human error, inconsistent data entry, or formatting differences. For example, "Rahul Sharma" and "R. Sharma" entered from two different

sources may refer to the same person, but no exact or field-level match will catch this. Detecting fuzzy duplicates requires advanced techniques such as phonetic matching, edit distance algorithms like Levenshtein distance, or machine learning based record linkage. They are most common when data is collected manually or aggregated from external sources with no standardized input format.

Question 5: Why should data validation be performed during transformation rather than after loading?

Answer:

Why Data Validation Should Be Performed During Transformation Rather Than After Loading

Data validation is the process of checking whether data meets defined rules, formats, and business requirements before it is considered fit for use. While validation can technically be applied at any stage of the ETL pipeline, performing it during the transformation phase rather than after loading is a fundamental principle of good pipeline design, and the reasoning behind this choice has both technical and business significance.

The most important reason is cost of correction. In software and data engineering, there is a well-established principle that the later an error is caught in a process, the more expensive it becomes to fix. If invalid data is allowed to pass through transformation and gets loaded into the destination database, correcting it requires identifying the bad records, writing correction scripts, re-running parts of the pipeline, and potentially notifying downstream teams or applications that consumed the incorrect data. All of this effort could have been avoided by rejecting or flagging the record at the transformation stage before it ever touched the target system.

The second reason is prevention of data poisoning in the target system. Once invalid data enters a data warehouse or reporting database, it immediately becomes available to dashboards, analysts, and automated reports. Any insights or decisions derived from that data during the window between loading and detection of the error are compromised. Validation during transformation acts as a gatekeeping layer that ensures only clean, rule-compliant data reaches the destination, keeping the target system trustworthy at all times.

The third reason is that transformation is contextually the right stage for validation. During transformation, the data is already being processed, business rules are being applied, and the structure is being reshaped. This is the natural point to check whether values fall within acceptable ranges, whether mandatory fields are populated, whether data types match the expected schema, and whether referential integrity holds. Bundling validation with transformation avoids the need for a separate post-load audit pass, making the pipeline more efficient and streamlined.

The fourth reason relates to error handling and routing. When validation is applied during transformation, rejected records can be redirected to a quarantine table or error log with detailed reasons for failure. This gives data engineers visibility into what went wrong and where, without affecting the main data flow. If validation is done after loading, bad

records are already mixed into the target table, making isolation and removal significantly more complex.

In conclusion, performing data validation during transformation is not just a best practice but a design necessity. It protects the integrity of the target system, reduces the cost and complexity of error correction, and ensures that downstream consumers always work with data that has been explicitly verified against business and technical rules before arrival.

Question 6: Explain how business rules help in validating data accuracy. Give an example.

Answer:

Data accuracy means that the values stored in a system correctly represent the real-world facts they are supposed to capture. While technical checks such as data type validation or null checks ensure structural correctness, they are not sufficient to confirm that data is meaningfully correct in a business context. This is where business rules play a critical role.

Business rules are conditions and constraints defined by an organization that reflect how its data should behave based on real-world logic. They go beyond format checks and impose domain-specific boundaries on what values are acceptable, how fields relate to each other, and what combinations of data are logically valid. When integrated into the transformation stage of an ETL pipeline, business rules act as intelligent filters that catch errors which would otherwise appear technically valid but are factually wrong.

Business rules validate accuracy in several ways. They enforce range constraints, ensuring values fall within realistic boundaries. They establish relational logic between fields, ensuring that values in one column are consistent with values in another. They apply conditional checks, meaning certain fields must satisfy specific conditions depending on the state of other fields. They also enforce referential and temporal logic, ensuring that dates, sequences, and relationships make chronological and operational sense.

Without business rules, technically clean data can still be factually wrong. A record may pass all format and type checks and still contain an age of 300 years, an order delivery date before the order was placed, or a discount greater than the product price. None of these errors would be caught by structural validation alone. Business rules are what bridge the gap between data that looks correct and data that actually is correct.

Example: Order Management Pipeline

Consider an e-commerce company running an ETL pipeline that processes daily order records. The following business rules are defined for validation during transformation.

The order amount must be greater than zero, because a completed order cannot have a zero or negative value. The discount applied must not exceed the original order amount, because giving more discount than the price itself is financially impossible. The order delivery date must be greater than or equal to the order placement date, because an item cannot be delivered before it is ordered. The customer age must be between 18 and 100, because the platform only allows adult users and ages beyond 100 are realistically invalid.

Now suppose a record comes in with an order placement date of 15th May 2024 and a delivery date of 10th May 2024. Structurally, both are valid date fields. But the business rule identifying that delivery cannot precede placement immediately flags this record as inaccurate and routes it to an error log for review.

Without this rule, the record would pass through, enter the dashboard, and cause the average delivery time metric to show a negative value, completely distorting operational reporting.

In conclusion, business rules are what give data validation its intelligence. They encode organizational knowledge into the pipeline and ensure that data is not just technically well-formed but operationally meaningful and accurate. A pipeline without business rule validation may produce clean data, but it cannot guarantee correct data.

Use the following Sales_Transactions dataset for all questions:

Txn_ID	Customer_ID	Customer_Name	Product_ID	Quantity	Txn Amount	Txn_Date	City
201	C101	Rahul Mehta	P11	2	4000	2025-12-01	Mumbai
202	C102	Anjali Rao	P12	1	1500	2025-12-01	Bengaluru
203	C101	Rahul Mehta	P11	2	4000	2025-12-01	Mumbai
204	C103	Suresh Iyer	P13	3	6000	2025-12-02	Chennai
205	C104	Neha Singh	P14	Null	2500	2025-12-02	Delhi
206	C105	N/A	P15	1	Null	2025-12-03	Pune
207	C106	Amit Verma	P16	1	1800	Null	Pune
208	C101	Rahul Mehta	P11	2	4000	2025-12-01	Mumbai

Question 7: Write an SQL query on Sales_Transactions to list all duplicate keys and their counts using the business key (Customer_ID + Product_ID + Txn_Date + Txn_Amount)

Answer:

SQL Query to Find Duplicate Business Keys

Business Key = Customer_ID + Product_ID + Txn_Date + Txn_Amount

```
SELECT
    Customer_ID,
    Product_ID,
    Txn_Date,
    Txn_Amount,
    COUNT(*) AS Duplicate_Count
```

```
FROM
    Sales_Transactions
```

```
GROUP BY
    Customer_ID,
    Product_ID,
    Txn_Date,
    Txn_Amount
```

```
HAVING
    COUNT(*) > 1;
```

Output Based on Given Dataset

Customer_ID	Product_ID	Txn_Date	Txn_Amount	Duplicate_Count
C101	P11	2025-12-01	4000	3

Explanation

GROUP BY combines all records sharing the same combination of the four business key fields into a single group.

COUNT(*) counts how many rows exist in each group.

HAVING COUNT(*) > 1 filters and returns only those groups where more than one row exists, meaning duplicates are present.

From the dataset, Txn_ID 201, 203, and 208 all belong to Customer C101, buying Product P11 on 2025-12-01 for ₹4000. All four business key fields are identical across these three records, making them exact duplicates with a count of 3.

Txn_IDs 202, 204, 205, 206, and 207 are unique combinations and therefore do not appear in the output.

Question 8 : Enforcing Referential Integrity Assume the following Customers_Master table:

CustomerID	CustomerName	City
C101	Rahul Mehta	Mumbai
C102	Anjali Rao	Bengaluru
C103	Suresh Iyer	Chennai
C104	Neha Singh	Delhi

Identify Sales_Transactions.Customer_ID values that violate referential integrity when joined with Customers_Master and write a query to detect such violations.

Answer:

Referential Integrity means every 'Customer_ID' in the 'Sales_Transactions' table must exist in the 'Customers_Master' table. If a transaction references a Customer_ID that is not present in the master table, it is a violation.

SQL Query to Detect Violations

```
SELECT
    st.Txn_ID,
    st.Customer_ID,
    st.Customer_Name,
    st.Product_ID,
    st.Txn_Date,
    st.Txn_Amount
FROM
    Sales_Transactions st
LEFT JOIN
    Customers_Master cm
ON
    st.Customer_ID = cm.CustomerID
WHERE
    cm.CustomerID IS NULL;
```


Output Based on Given Data

Txn_ID	Customer_ID	Customer_Name	Product_ID	Txn_Date	Txn_Amount
206	C105	N/A	P15	2025-12-03	Null
207	C106	Amit Verma	P16	2025-12-02	1800